Arquitetura Von Neumann

Monday, April 20, 2026 1:57 PM

1. Simplicidade de Hardware e Custo

Na arquitetura de von Neumann, existe apenas **um único espaço de memória principal** e um **único barramento** (caminho de comunicação) compartilhado para buscar tanto instruções quanto dados.

- **Vantagem:** Isso torna o design da placa-mãe e do processador muito mais simples e barato de fabricar em massa. Você não precisa de trilhas de hardware separadas para o código e para os dados (como ocorre na arquitetura de Harvard).

2. Flexibilidade e Propósito Geral

Como o código é tratado exatamente da mesma forma que os dados, a memória pode ser alocada de forma dinâmica.

- **Vantagem:** Se você está rodando um programa pesado (como um jogo ou um compilador C), a maior parte da memória pode ser preenchida com instruções. Se está processando um banco de dados gigante, a mesma memória pode ser quase inteiramente dedicada a dados. A máquina se adapta ao software, e não o contrário.

3. Facilidade para o Sistema Operacional

Tratar código como dado facilita imensamente o trabalho do Sistema Operacional. Quando você executa um binário no Linux, por exemplo, o SO simplesmente lê o arquivo do disco (como se fosse um dado qualquer) e o copia para a RAM. A partir daí, ele aponta o *Instruction Pointer* (o registrador RIP no x86-64) para o início desse bloco de memória e a CPU começa a executar.

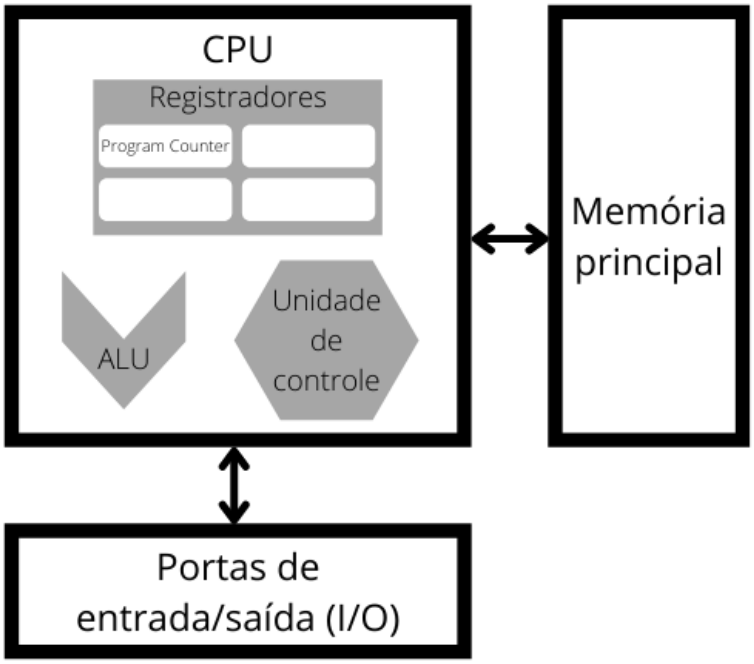
4. Segurança e Exploits

Do ponto de vista de segurança ofensiva e baixo nível, a arquitetura de von Neumann é a razão exata pela qual muitas vulnerabilidades clássicas existem. Como a CPU não sabe distinguir intrinsecamente o que é uma instrução Assembly do que é um dado puro (ambos são apenas bytes na mesma memória), técnicas como **Buffer Overflows** são possíveis. Se você conseguir sobrescrever um espaço de memória destinado a dados (como variáveis) com o seu próprio *shellcode* e redirecionar o fluxo de execução para lá, a CPU vai executar aquele dado malicioso como se fosse um código legítimo da aplicação.

Arquitetura x86

Monday, April 20, 2026 2:01 PM

A arquitetura x86 segue o modelo da arquitetura de Von Neumann (instruções e dados na compartilham memória) onde esse, mais uma vez resumindo, trabalha principalmente usando uma unidade central de processamento (CPU) e uma memória principal.



As instruções podem trabalhar manipulando/lendo dados em registradores que são pequenas áreas de memória internas à CPU. E também pode manipular dados na memória principal que no caso é a memória RAM. Bem como também usar o sistema de entrada e saída de dados, feito pelas portas físicas.

O registrador *Program Counter* no diagrama acima armazena o endereço da próxima instrução que será executada na memória principal. Na arquitetura x86 esse registrador é chamado de *Instruction Pointer*.

Modos de Operação

Monday, April 20, 2026 2:23 PM

Um processador x86-64 consegue executar código de versões anteriores simplesmente trocando o modo de operação. Cada modo faz com que o processador funcione de maneira um tanto quanto diferente, fazendo com que as instruções executadas também tenham resultados diferentes.

Barramento interno

Os tais "bit" que são muito conhecidos mas pouco entendido, na verdade é simplesmente uma referência a largura do barramento interno do processador quando ele está em determinado modo de operação. A largura do barramento interno do processador nada mais é que o tamanho padrão de dados que ele pode processar de uma única vez.

Tradução e Compilação

Monday, April 20, 2026 2:34 PM

Quando você passa o seu código pelo **Assembler** (como o NASM ou o GAS), o resultado não é um executável pronto para rodar, mas sim um **Arquivo Objeto** (geralmente com extensão .o ou .obj).

Header (Cabeçalho)

- Ele diz ao sistema operacional e ao Linker como interpretar o restante do arquivo.

Segmento de Texto (Text Segment)

- Instruções traduzidas, **Opcodes** (códigos de operação), que a CPU entende.
- As instruções (`mov`, `add`, `push`).

Segmento de Dados Estáticos (Static Data Segment)

- Este segmento armazena variáveis que têm um tempo de vida equivalente ao do próprio programa.
- As variáveis globais (`.data` e `.bss`).

Informações de Relocação (Relocation Info)

- Alguns conteúdos dependem de referências absolutas no programa carregado

Tabela de Símbolos

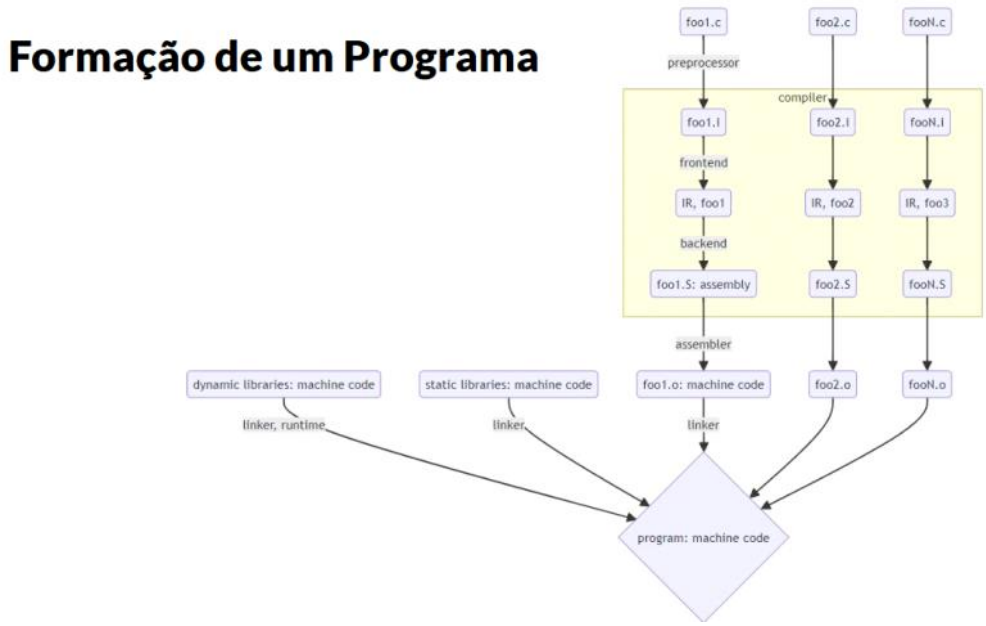
- Funções ou variáveis globais inicializadas.
- Referências Externas

Informações de Debug

- São a ponte final entre os bits e bytes do arquivo objeto e o código que você escreveu originalmente. Sem elas, o depurador (como o GDB ou o x64dbg) veria apenas endereços de memória e instruções Assembly, sem saber a que linha de C ou de qual arquivo fonte elas pertencem.
- Associação com o código fonte (se for o caso)

Formação de um programa

Monday, April 20, 2026 3:48 PM



O exemplo da imagem descreve o processo padrão de um programa escrito em **C** (ou C++), que é uma linguagem compilada.

- O arquivo fonte é o arquivo de texto plano que o programador, escreve. Ele contém as instruções na linguagem humana (C, no caso).
- Os arquivos fonte são os que terminam em `.c` (`foo1.c`, `foo2.c`

O Fluxo de Formação de um Programa

O processo transforma código legível em algo executável:

1. **Pré-processador:** Limpa o código (resolve `#include`).
2. **Compilador:** Traduz C para **Assembly** (linguagem intermediária).
3. **Assembler (NASM/GAS):** Traduz Assembly para **Código de Máquina** (Arquivo Objeto).
4. **Linker:** O grande finalizador. Ele junta vários arquivos objeto, resolve as referências da Tabela de Símbolos e anexa bibliotecas (estáticas ou dinâmicas) para criar o **Executável Final**.

A Fase do Compilador (O retângulo amarelo)

Aqui, cada arquivo fonte (como `foo1.c`) é tratado individualmente.

- **Preprocessor:** O primeiro passo. Ele resolve as diretivas que começam com # (como `#include` ou `#define`). O resultado é o arquivo `.i` (código C puro expandido).
- **Frontend:** Analisa a gramática do código C e verifica se você não esqueceu nenhum ponto e vírgula ou errou algum tipo. Ele gera o **IR (Intermediate Representation)**, que é um código genérico, independente do processador.
- **Backend:** É aqui que a mágica da arquitetura acontece. O backend traduz aquele código genérico (IR) para o **Assembly** específico do seu processador (x86, ARM, etc.), gerando os arquivos `.S`.

Ligador (Linker)

Monday, April 20, 2026 3:58 PM

1. Produz uma “Imagem Executável”

O termo "imagem" é usado porque o arquivo final (ELF no Linux, PE no Windows) é um reflexo fiel de como os dados serão mapeados na RAM. O Linker organiza o arquivo de modo que o Sistema Operacional saiba exatamente:

- "Este pedaço vai para o endereço X e é apenas leitura (código)."
- "Este pedaço vai para o endereço Y e permite escrita (variáveis)."

2. Junta Segmentos

Vários arquivos objeto (foo1.o, foo2.o, etc.), cada um tinha seu próprio segmento de **Texto** e de **Dados**.

- O Linker pega todos os segmentos de texto de todos os arquivos e os agrupa em um único e grande **Segmento de Texto** do programa final.
- Ele faz o mesmo com os segmentos de dados. Isso organiza o programa de forma lógica para a CPU.

3. Resolve Rótulos (Ex: Instruções de Desvio)

No Assembly, você usa rótulos (labels) como jmp loop ou call calcular.

- No arquivo objeto, o Assembler não sabe o endereço real do loop ou do calcular. Ele deixa um espaço em branco.
- O Linker "resolve" isso: ele calcula a distância em bytes ou o endereço absoluto final e substitui o nome do rótulo pelo número (endereço) correto. Agora a CPU sabe exatamente para qual byte saltar.

Normalmente, a CPU executa uma instrução após a outra (sequencialmente). As instruções de desvio dizem à CPU: *"Ei, em vez de ir para a próxima linha, pule para este outro endereço de memória aqui!"*.

4. Corrige referências dependentes de local e externas

Este é o ponto mais crítico da ligação e se divide em dois problemas:

- **Referências Externas:** Se o foo1.c usa uma função que está no foo2.c, o Linker encontra o endereço da função no foo2.o e o escreve dentro do código do foo1.o.
- **Referências Dependentes de Local (Relocação):** Muitos códigos precisam saber seu próprio endereço absoluto para funcionar. Se o Linker decide que o segmento de dados vai começar no endereço 0x600000, ele precisa passar por todo o código de máquina "corrigindo" todas as instruções que apontavam para endereços de dados, atualizando-as para esse novo local real.

Loader (Carregador)

Monday, April 20, 2026 4:45 PM

1. Carrega imagem do disco para a memória

O SO localiza o arquivo executável no armazenamento (HD ou SSD). Ele não simplesmente "joga" o arquivo na RAM, mas começa a mapear a estrutura que o Linker criou (a "imagem") para o espaço de trabalho real da CPU.

2. Lê o Header para obter tamanho dos segmentos

Antes de qualquer coisa, o SO lê os primeiros bytes do arquivo (o **Header**).

- **O que ele procura:** "Qual o tamanho do Segmento de Texto?", "Quanto de RAM preciso reservar para o Segmento de Dados?".
- **Importância:** Sem isso, o SO não saberia quanta memória pedir ao hardware para rodar o programa.

3. Cria espaço de endereçamento virtual

Esta é uma das funções mais geniais dos Sistemas Operacionais modernos. O SO cria uma "ilusão" para o programa.

- **A Ilusão:** O programa acha que tem toda a memória RAM do computador só para ele, começando geralmente do endereço 0x0000....
- **A Realidade:** O SO mapeia esses endereços virtuais para endereços físicos reais na RAM. Isso impede que um programa acesse ou "atropele" a memória de outro programa.

4. Copia segmento de texto e inicializa dados

- **Segmento de Texto:** As instruções de máquina são copiadas do disco para a RAM. Esse bloco é marcado como **"Apenas Execução/Leitura"** para evitar que o programa se altere.
- **Dados Inicializados:** O SO copia as variáveis que já têm valor (ex: int x = 10;).
- **Dados Não Inicializados (.bss):** O SO apenas reserva o espaço e o preenche com **zeros**. É por isso que variáveis globais não inicializadas em C começam com valor zero.

5. Coloca segmentos na pilha (Stack)

O SO prepara a **Stack (Pilha)**, que é essencial para a execução:

- **Argumentos e Ambiente:** Ele coloca na pilha os argumentos de linha de comando (o famoso argc e argv do C) e as variáveis de ambiente.
- **Endereço de Retorno:** Prepara a estrutura para que as funções possam ser chamadas e saibam para onde voltar.

6. Inicializa Registradores

Antes de começar, o SO limpa ou configura os registradores da CPU para que o programa comece em um estado conhecido e seguro.

- **Ponteiro de Pilha (RSP/ESP):** O SO aponta este registrador para o topo da pilha que ele

acabou de criar. Sem isso, o programa não conseguiria nem fazer a primeira chamada de função.

- **Registradores de Propósito Geral:** Geralmente são zerados, mas alguns podem conter informações iniciais (como o endereço da tabela de argumentos).
- **Flags:** O registrador de status (EFLAGS) é resetado para garantir que não existam "restos" de cálculos de programas anteriores (como um sinal de "vai um" ou erro).

7. Salta para a rotina inicial (Entry Point)

Este é o momento crítico onde o SO deixa de ser o protagonista e o seu código assume o comando.

- **O que é o Entry Point?** É o endereço de memória da primeiríssima instrução do programa. No cabeçalho (Header) do arquivo, o Linker deixou gravado: "O código começa no endereço `0x401000`".
- **O "Salto":** O SO carrega esse endereço no **Instruction Pointer (RIP/EIP)**.
- Como a função da CPU é sempre "executar o que o IP aponta", no instante em que o IP recebe esse valor, a CPU busca a primeira instrução do seu programa na RAM e a executa.

CPU X RAM

Monday, April 20, 2026 2:04 PM

A maneira mais direta de entender a diferença entre CPU e RAM, especialmente quando você está lidando com linguagens de baixo nível como C ou Assembly, é pensar na divisão entre **ação** e **armazenamento de curto prazo**.

1. CPU (Unidade Central de Processamento)

A CPU é o "motor" ou a unidade ativa do sistema. Ela não armazena grandes quantidades de dados; o objetivo dela é calcular, tomar decisões lógicas e mover dados de um lugar para outro.

- **O que contém:** Circuitos lógicos (ALU para matemática), uma Unidade de Controle e **Registradores** (como RAX, RBP, RIP na arquitetura x86-64). Os registradores são a memória mais rápida do computador, operando na mesma velocidade do processador, mas há pouquíssimos deles.
- **O que faz:** Fica eternamente em um ciclo de *Fetch-Decode-Execute* (Buscar, Decodificar, Executar). Ela busca uma instrução (opcode), descobre o que ela significa e a executa.
- **Inteligência:** É "burra" no sentido de que faz exatamente o que a instrução manda. Se o registrador de instrução (RIP) apontar para um lixo na memória, a CPU vai tentar executar esse lixo como se fosse código legítimo.

2. RAM (Memória de Acesso Aleatório)

A RAM é uma unidade passiva. Você pode imaginá-la como um array (vetor) gigantesco de bytes, onde cada byte tem um endereço único (como `0x7fffc000`).

- **O que contém:** Tudo o que precisa estar disponível instantaneamente para o programa rodar. Seguindo o modelo de von Neumann, a RAM armazena tanto as instruções compiladas do seu programa (o segmento de `.text`) quanto os dados que ele usa (como a `.bss`, a *Heap* para alocação dinâmica e a *Stack* para variáveis locais).
- **O que faz:** Apenas guarda informações e as entrega ou atualiza quando a CPU solicita.
- **Volatilidade:** É temporária. Se faltar energia, os capacitores descarregam e tudo o que estava lá (seu código em C rodando, o sistema operacional, os buffers) é perdido.

Características Essenciais

- **Acesso Aleatório:** O termo "Random Access" significa que a CPU pode acessar qualquer endereço de memória (seja o primeiro byte ou o último) com a mesma velocidade. Não é necessário ler a memória em sequência.
- **Volatilidade:** Ela é uma memória de trabalho temporária. Sem energia elétrica, todos os dados nela contidos desaparecem. É por isso que você "perde o que não salvou" se o computador desligar.
- **Velocidade de Barramento:** Ela é muito mais rápida que o armazenamento (SSD/HD), permitindo que a CPU não fique esperando tempo demais para receber as instruções do seu código.